

Amphibious Locomotion of Polymander

A Salamander-like Robot

Contents

1	Introduction	1
1.1	Embodiment	1
1.2	Code Structure	3
1.3	Report Structure	3
1.4	Submission	3
2	Assignment	5
2.1	Wave Controller and Metrics	5
2.2	CPG controller	7
2.3	Sensory feedback	11

1 Introduction

In earlier practicals, we learned the basics of numerical integration, simulated a pendulum with passive properties, and modeled oscillator dynamics of a 2-neuron network. This project aims to combine and extend those concepts to implement a more comprehensive controller based on Central Pattern Generators (CPGs) and deploy it on a salamander-like robot - Polymander - to achieve aquatic and terrestrial locomotion.

A realistic neuromechanical model integrates neural control with the biomechanical properties of robot body and its interaction with the fluid environment. Such a model allows us to systematically test how different neural parameters affect locomotion performance.

In **Project 1**, you will focus on modeling and simulating the swimming behavior of the robot. You will start with a simple wave controller and implement the evaluation metrics. You will then design a CPG controller to propel the robot to swim. In the end, you will add sensory feedback to the CPG network and study its role in the robot locomotion.

In **Project 2**, you will extend this controller to the walking gait.

1.1 Embodiment

Polymander [1] is a reconfigurable amphibious fish-like robot that could mimic both the *Polypterus* (a fish that can walk on land and has a bipedal body plan) and *Salamander* (quadruped body plan). In this project, we are using its quadruped body plan and focusing on its representation of *Salamander*. Polymander consists of 11 modular axial body segments, including a head, a front girdle and a hind girdle, six axial modules, and a tail module of two parts. For each girdle, it supports two attachable limbs made of a leg module and a foot module ending in a silicone ball-shaped foot.

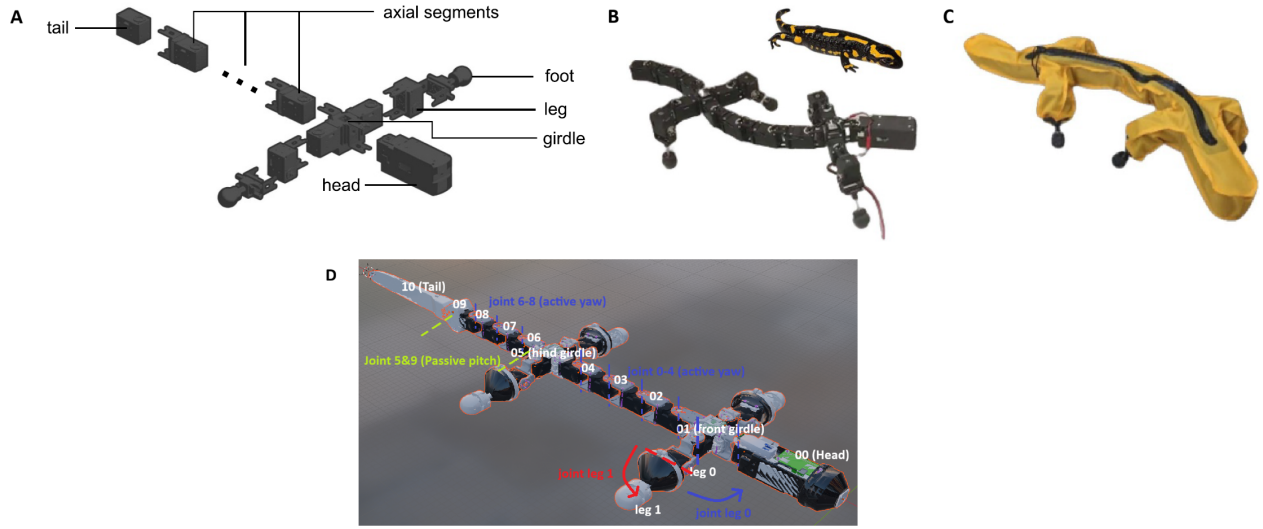


Figure 1: Polymander Robot. (A) Reconfigurable modules of the robot. (B) Salamander configuration with two forelimbs and two hindlimbs. (C) Polymander inside its waterproof suit. (D) Links and joints definition of Polymander in the quadruped plan.

Along the body axis, the Polymander model consists of $n_{links} = 11$ links connected by $n_{joints} = 10$ rotational joints. The joints are numbered from 0 to 9, ordered from the head to the tail. Besides joint 5 and joint 9 that are passive pitch joints moving up and down, all other joints are actively controlled yaw joints that move left and right. For each limb, it has one active joint connecting the girdle and the leg module that moves forward/backward (abduction/adduction) and one active joint connecting the leg module and the foot module that swings downward/upward (flexion/extension).

To imitate the swimming behavior of real animals, the torque produced by the servo motors at each actively controlled joint is computed by means of Ekeberg muscle models (that will be explained in a lecture). For each joint, the model receives in input the left (M_{L_i}) and right (M_{R_i}) activations from the muscle cells, the current joint angle (θ_i) and joint speed ($\dot{\theta}_i$) to compute the resulting output torque (τ_i) via:

$$\tau_i = \alpha_i M_{diff_i} + \beta_i (\gamma_i + M_{sum_i}) \theta_i + \delta_i \dot{\theta}_i$$

$$M_{diff_i} = (M_{L_i} - M_{R_i})$$

$$M_{sum_i} = (M_{L_i} + M_{R_i})$$

Where α_i is the active gain, γ_i is the passive stiffness, β_i is the active stiffness, δ_i is the damping coefficient. The Ekeberg muscle model is a rotational spring-damper system with the addition of a variable stiffness term $\beta(M_L + M_R)\theta_i$. The neutral position of the stiffness is fixed at zero for spine joints and at an extended position for limb joints for Project 1. The active term of the model acts an external torque $\alpha(M_L - M_R)$. You do not need to optimize the muscle parameters, and `muscle_params.csv` contains the tuned muscle parameters ready for use. We assume robot motors in the simulation to be ideal and do not account for real motor non-linearities and dynamics.

When submerged in water, the body is subjected to buoyancy and hydrodynamic forces. Buoyancy forces are applied to each link according to its submerged volume V_{UW}

$$F_{b_i} = V_{UW_i} \rho g$$

Where ρ is the water density and g is the gravity acceleration ($g = 9.81m/s^2$). We consider the swimming of the robot in high Reynolds number regimes and opted for a simplified hydrodynamic model composed of inertial drag forces. For each link, a speed-dependent drag force (F_d) is applied in each dimension (i) according to:

$$F_{d_i} = \frac{1}{2}\rho C_{d_i} A_i V_i^2$$

Where A is the link's cross-sectional area, c_{d_i} is the drag coefficient in the i -th dimension, V_i is the current speed of the link in the i -th dimension in the reference frame of the center of mass of the link. The drag coefficients were chosen to match the swimming speed of Polymander in real life in its swimming suit when replaying the swimming kinematics. You do not need to tune the drag coefficients, and they are pre-defined in the model.

1.2 Code Structure

The code is hosted at <https://gitlab.com/farmsim/courses/cmc-2026-students>. The README.md gives details on the installation. The files you need to change are marked in red and the lines to complete are tagged with #TODO in each part of the assignment. Generated outputs (controller states, evaluation metrics, figures, videos) for each assignment should be saved to logs/exercise_{assignment number} folder.

Optionally, you could create plotting utilities in cmc_controllers/plot_utils.py and use them in the main assignment files. Unless necessary and well motivated, no additional dependencies shall be added to the environment.

1.3 Report Structure

Include a short report including plots and results addressing the questions stated under each assignments (see below). You do not need to repeat the methodology that already exists in this file, rather your own methods and results that you generated. The report has to be no longer than 8 pages, excluding references, generative AI declaration and supplementary material. For supplementary material, we allow 2 additional pages in case you have extra diagrams to show but the main plots and graphs should be in the main report.

1.4 Submission

Deadline: The Project 1 needs to be handed in by **Wednesday, 29.04.2026 at 23:59.**

When submitting, zip your report and your code together as cmc_project1_group_{group_number}.zip, using your group number in the filename. Your code should include code files only. The virtual environment (e.g. the venv directory), the farms folder, and the logs folder should not be included. Videos and presented results in the report should be reproducible by running the main exercise scripts. Videos should have assignment numbers as prefixes and descriptive phrases as suffixes that you refer to in your report. The structure of your submission folder should look like below. The files marked with * are the files to modify in the project.

```
cmc_mp1_group_{group_number}_name1_name2_name3.zip*
|-- cmc_mp1_group_{group_number}_name1_name2_name3.pdf*
\-- cmc_controllers/
    |-- neural_data.py
    |-- neural_options.py
    |-- polymander_controller.py
    |-- metrics.py*
    |-- wave_controller.py*
    \-- CPG_controller.py*
|-- exercise1_1.py*
|-- exercise1_2.py*
|-- exercise2_1.py*
|-- exercise2_2.py*
|-- exercise2_3.py*
|-- exercise3_1.py*
|-- exercise3_2.py*
|-- exercise3_3.py*
\-- videos*
    \-- ...
```

Pro tips

- **Check the notes on the code.** Many explanations are given therein.
- **Be concise and precise** in the answers to the exercises. Do not repeat the methodology existing in this file.
- Back up your conclusion with **videos** and **Figures** to make your reasoning compact and strong.

2 Assignment

2.1 Wave Controller and Metrics

Before implementing the CPG controller with couplings, we start by implementing a basic wave (i.e. sine) controller that generates a periodic traveling wave along the axial joints of the robot. The wave controller is designed with the intuition that the robot joints exhibit a traveling wave during swimming behaviors. When the active torque of the muscle dominates, the wave-like torque would likely induce acceleration, velocity, and joint pose close to a wave-like pattern that would propel the robot forward.

The wave controller is implemented such that:

- All axial joints maintain the same cocontraction level of 2.0 during swimming as in Eq.1
- Active torques generated by axial joints form a periodic traveling wave as in Eq.2.

$$\begin{aligned} M_{sum_i}(t) &= M_{L_i}(t) + M_{R_i}(t) \\ &= 2 \end{aligned} \quad (1)$$

$$\begin{aligned} M_{diff_i}(t) &= M_{L_i}(t) - M_{R_i}(t) \\ &= A \cdot \sin\left(2\pi \left(ft - \epsilon \frac{i}{N_{joints}}\right)\right) \end{aligned} \quad (2)$$

where ϵ is the total wave lag, A is the amplitude and f is the frequency of the wave.

Question 1.1 Complete your implementation of `WaveController()` in `cmc_controllers/wave_controller.py` marked with **TODO**. Run `scripts/exercise_1.1.py` to initiate an instance of your wave controller with $A = 2$, $f = 2Hz$, $TWL = 1$ to use it to simulate the robot swimming. Set `recording` to `True` and record a video of the swimming behavior.

To analyze and study the locomotion behavior of the robot under different controller parameters. We define two kinds of metrics to quantify the simulation result in `cmc_controllers/metrics.py`. The **Neural Metrics** (NM) quantifies the neural signals generated by the controller (we consider the difference between left and right muscle activation to be the neural signal per joint). The **Mechanical Metrics** (MM) quantifies how effectively and efficiently the body segments/joints move.

We define the following metrics in `metrics.py`.

- **NM1:** *Neural frequency* (f_{neur}) and *Neural amplitude* (A_{neur}):

The frequency and the amplitude of a neural signal are computed via a Fast Fourier Transform (FFT).

$$\begin{aligned} f_{neur} &= \underset{f}{\operatorname{argmax}} |X_i[f]| >_i \\ A_{neur} &= \underset{N}{\operatorname{argmax}} \frac{2 |X_i[k_{max}^i]|}{N} >_i \end{aligned}$$

where $|X_i[k_{max}^i]|$ is the magnitude of the FFT at the index corresponding to the dominant frequency for each signal, and N is the number of samples.

- **NM2:** *Intersegmental phase lag (IPL_{neur}):*

Intersegmental phase lag is used to quantify the phase difference among oscillating signals from different body segments. For two sinusoidal signals oscillating at similar frequencies, the delay $\Delta t_{1,2}$ is computed by cross-correlation:

$$\Delta t_{1,2} = \underset{\tau}{\operatorname{argmax}} (\operatorname{corr}(x_1, x_2))$$

The corresponding phase lag is computed by normalizing the delay by the average frequency of the two signals $f_{1,2}$, as:

$$PL_{1,2} = 2\pi f_{1,2} \Delta t_{1,2},$$

Finally, the overall intersegmental phase lag is computed by averaging the phase lag values for each successive couple of signals along the body:

$$IPL_{neur} = \frac{1}{N-1} \sum_{i=0}^{n-2} PL_{i,i+1}$$

Note: The phase lag of non-neural signals (e.g. mechanical joint angles) can be computed similarly.

- **MM1:** *Mechanical frequency (f_{mech}) and amplitude (A_{mech}).*

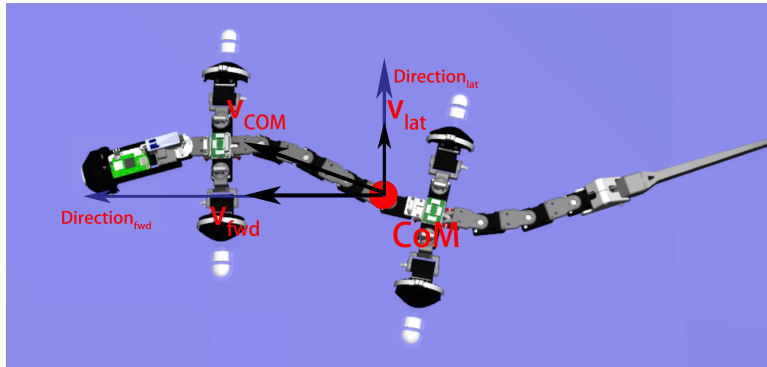
These metrics are computed analogously to their neural counterparts, considering the joint angles instead of the muscle commands.

- **MM2:** *Forward speed ($v_{fwd}, v_{fwd}(t)$) and lateral speed ($v_{lat}, v_{lat}(t)$).*

At each time step, we project the center-of-mass velocity $\mathbf{v}_{COM}(t)$ onto the forward ($d_{fwd}(t)$) and lateral ($d_{lat}(t)$) principal axes of the robot. The overall speed metrics are computed by averaging the instantaneous values over time:

$$v_{fwd} = \langle \mathbf{v}_{COM}(t) \cdot \mathbf{d}_{fwd}(t) \rangle_t$$

$$v_{lat} = \langle \mathbf{v}_{COM}(t) \cdot \mathbf{d}_{lat}(t) \rangle_t$$



- **MM3:** *Curvature κ*

Curvature is used to evaluate the straightness of the robot trajectory. For a trajectory in 2D, the curvature is

$$\kappa = \frac{\dot{x}\ddot{y} - \ddot{x}y}{(\dot{x}^2 + \dot{y}^2)^{3/2}}$$

where $\dot{x}$ and $\dot{y}$ are the speed components.

A large curvature indicates a sharp turning direction, while a small curvature indicates a straighter line. The sign indicates the turning direction.

- **MM4:** *Energy consumption (E) and Cost of transport (CoT).*

We compute energy as an integration of the positive power $\tau_j(t)\dot{\theta}_j(t)$ at each joint. We do **not** model energy recovery from negative work.

$$P_j(t) = \max(\tau_j(t)\dot{\theta}_j(t), 0)$$

$$E = dt \times \sum_t \sum_{j=1}^n P_j(t)$$

The cost of transport relates the energy expenditure to the distance traveled.

$$\text{CoT} = \frac{E}{D_{\text{fwd}}},$$

where D_{fwd} is the forward distance covered by the center of mass (CoM).

Question 1.2 Complete the implementation of metrics **NM2** (IPL_{neur}), **MM2** (v_{fwd} and v_{lat}), **MM4** (CoT and E) in `cmc_controllers/metrics.py`. Modify `post_processing()` in `cmc_controllers/exercise_1.1.py` to load the simulation result from your wave controller and evaluate its performance. Do they match your expectations? Plot the time evolution of joint angles for a few cycles and the CoM trajectory of the robot during swimming in 2D.

Question 1.3 Explore the effect of different wave controller parameters on the performance metrics of the robot locomotion in `cmc_controllers/exercise_1.2.py`. Simulate the robot swimming and perform a 2D-grid parameter investigation of $A \in [1.0, 4.0]$, $TWL \in [0.2, 1.5]$ and $f = 1.5\text{Hz}$. Qualitatively analyze the effect on the forward speed, CoT, average IPL_{neur} , and average ϕ_{lock} .

Hint: For running several controllers with different hyperparameters in parallel, you could use `run_multiple()`. Adjust max workers depending on your hardware capacity. This function would be useful in later exercises.

2.2 CPG controller

Here, we first describe the equations of the CPG network you will implement to control the Polymander robot (replacing the wave controller of the previous section). Each active joint (body and limb) is governed by a pair of oscillators representing the extensor and flexor in real muscle configuration. In total, we consider a double-chain oscillator with $n_{\text{cpg_body}} = 16$ oscillators along the body and $n_{\text{cpg_limb}} = 16$ oscillators distributed over 4 limb. During swimming, the limb network is considered to be saturated with zero amplitude and **we consider only the axial network in Project 1**. An illustration of the network is shown as in Fig 2.

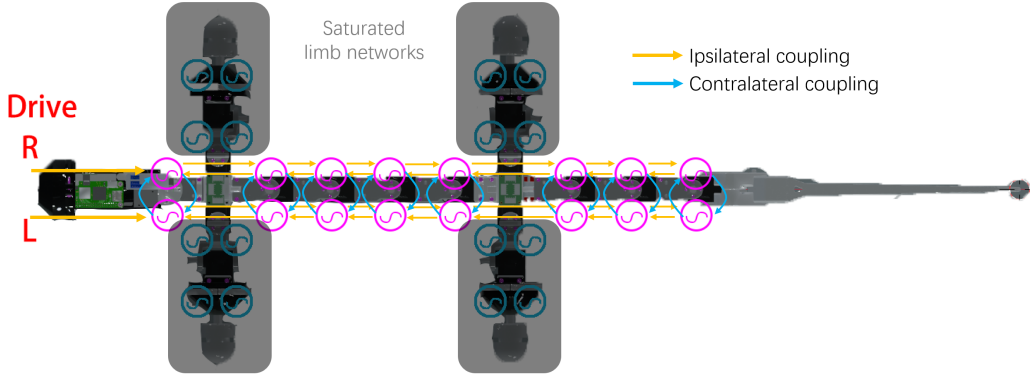


Figure 2: Configuration of the spinal CPG controller in Polymander. Limb networks are saturated and not considered in Project 1. Axial active joints are driven by 8 pairs of Ekeberg muscles receiving motor outputs from a double chain of 16 oscillators with ipsilateral and contralateral coupling. The CPG model receives descending signals from the drive representing the brain MLR region in real animals.

The oscillator's phase equation is defined in Eq.3, with θ_i the oscillator phase, f_i the oscillator intrinsic frequency, w_{ij} the coupling weights, ϕ_{ij} the nominal phase lag (phase bias), r_i the oscillator amplitude.

$$\dot{\theta}_i = 2\pi f_i + \sum_j r_j w_{ij} \sin(\theta_j - \theta_i - \phi_{ij}) \quad (3)$$

In this model, we only consider couplings between ipsilateral (i.e. successive same-side oscillators) and contralateral (i.e. left-right) oscillators as in Eq. 4.

$$w_{ij} = \begin{cases} w_{body2body_rostral}, & \text{if } i,j \text{ are adjacent oscillators from tail to head} \\ w_{body2body_caudal}, & \text{if } i,j \text{ are adjacent oscillators from head to tail} \\ w_{body2body_contra}, & \text{if } i,j \text{ are opposite oscillators on the other side} \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

The nominal phase lags are defined in Eq. 5, such that the phase lags between contralateral segments are constant π and the phase lags between ipsilateral segments are constant values $PB_{i,j}$ that sum up to ϕ_{body_total} from head to tail.

$$\phi_{ij} = \begin{cases} PB_{i,j}, & \text{if upwards (ipsilateral)} \\ -PB_{i,j}, & \text{if downwards (ipsilateral)} \\ \pi, & \text{if left to right (contralateral)} \\ -\pi, & \text{if right to left (contralateral)} \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

where

$$\sum_{i=0}^{n_{joint}-1} PB_{i,j} = \phi_{body_total}$$

The oscillator's amplitudes are defined in Eq. 6, with r_i the oscillator amplitude, R_i the nominal amplitude and a_i the convergence rate of nominal amplitudes. Note that this equation is actually the same equation as in [2] and has been simplified into a first-order ODE in order to simplify the implementation in this project.

$$\dot{r}_i = a_i(R_i - r_i) \quad (6)$$

The oscillator frequency (Eq. 7) and nominal amplitudes per joint (Eq. 8) are defined in Eq. 8 as linear functions of drive d with an offset and a lower/upper saturation bound as in [2] to simulate that the CPG controller can receive descending commands from a higher level controller (brain commands in the real animal). Similarly to the real animal, an increasing descending drive tends to increase both the frequency and amplitude of muscle contractions (within some saturation limits d_{low} and d_{high}) and therefore faster swimming.

$$f_i = \begin{cases} 0 & \text{if } d < d_{low} \text{ or } d > d_{high} \\ G_{freq_i} \cdot (d - d_{low}) + c_{f_i,0}, & \text{Otherwise} \end{cases} \quad (7)$$

$$R_i = \begin{cases} 0 & \text{if } d < d_{low} \text{ or } d > d_{high} \\ G_{amp_i} \cdot (d - d_{low}) + c_{R_i,0}, & \text{Otherwise} \end{cases} \quad (8)$$

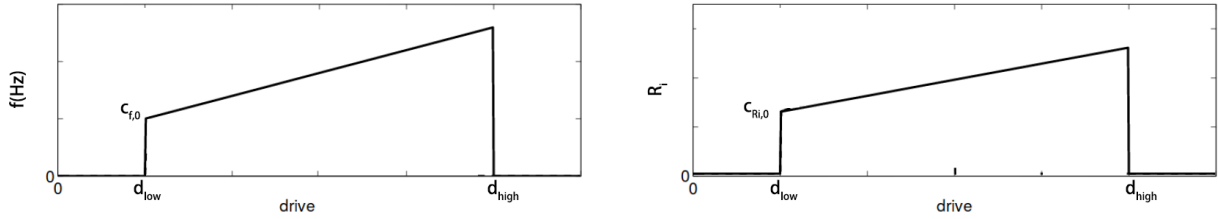


Figure 3: Axial oscillator properties as a function of drive. Reference: [2]

The oscillator phases and amplitudes evolve with Euler numerical integration as in Eq. 9.

$$\begin{aligned} r_{i,t+1} &= r_{i,t} + \dot{r}_{i,t} \cdot \Delta t \\ \theta_{i,t+1} &= \theta_{i,t} + \dot{\theta}_{i,t} \cdot \Delta t \end{aligned} \quad (9)$$

Finally, the muscle output of an oscillator is defined as a function of oscillator phase and amplitude in Eq. 10.

$$M_i = r_i(1 + \cos(\theta_i)) \quad (10)$$

Question 2.1 Complete `cmc_controller/CPG_controller.py` to implement the CPG controller with the default parameters in Tab. 1. Modify the `__init__` function to initialize the CPG nominal amplitudes, frequencies, coupling weights, and phase biases. Modify the `network_ode` and `step` function to implement the ODE and derivative updates. Test your implementation by running the network using `cmc_controllers/exercise2.1.py`. Record a video of the working controller with fixed drives of $d_{left} = d_{right} = 3$. Plot the time evolution of oscillator states (θ and r), sum and difference of muscle outputs ($M_{L_i} + M_{R_i}$), ($M_{L_i} - M_{R_i}$), body joint angles, and robot CoM trajectory in 2D for the first 5s of the simulation.

Hint: For oscillator states that need frequent updates (e.g. amplitudes and their derivatives), initialize a `state` array and compute the ODE update of `state` array in `network_ode` function. For oscillator parameters that are fixed once initiated (e.g. coupling weights), compute them once in `init` and retrieve later in `network_ode` function.

Parameter	Unit	Defaults
d_{low}	Lower threshold for drive saturation	1
d_{high}	Higher threshold for drive saturation	5
a_i	Convergence rate of nominal amplitudes in 1/s	3 for all
$c_{f_i,0}$	Offset of the oscillator frequency in Hz	1 for all
$c_{R_i,0}$	Offset of nominal amplitude per joint	0.5 for all
G_{freq_i}	Gain of the drive-dependent frequency per oscillator	0.5 for all
G_{amp_i}	Gain of the drive-dependent nominal amplitude per oscillator	0.25 for all
$PB_{i,j}$	Nominal phase bias between $joint_i$ and $joint_j$	$\frac{2\pi}{n_{joint}}$ for all
$w_{body2body_rostral}$	Ipsilateral rostral coupling strength	5
$w_{body2body_caudal}$	Ipsilateral caudal coupling strength	5
$w_{body2body_contra}$	Contralateral coupling strength	10
ϕ_{i0}	Initial state for oscillators	$random_{seed42}(0, 2\pi)$

Table 1: Default values for CPG controller parameters

Question 2.2 Explore the effect of drive parameters and body phase bias in `cmc_controllers/exercise2.2.py`. Run simulations with a 2D grid search of $d = d_{left} = d_{right} \in [2.0, 4.0]$ and $PL \in [\frac{\pi}{2n_{joint}}, \frac{3\pi}{n_{joint}}]$ rad, with other parameters at their default values in Tab. 1. How do drives and phase biases affect the robot's CoT and forward speed? Run simulations with a 2D grid search of $d_{left} \in [2.0, 4.0]$ and $d_{right} \in [2.0, 4.0]$. How does the differential drive affect the curvature and the trajectory of the robot?

So far, we have empirically set default parameters to initiate a CPG controller. We want to compare the controller performance with the swimming for a real animal.

We have collected a 2D swimming sequence from a *Salamander Pleurodeles Waltl* as shown in Fig 4. The subject was implanted with 11 video-opaque XROMM [3] markers along the spine under its skin and was recorded under a high-speed X-ray camera during swimming (credit: Anthony Herrel). The 11 recorded markers were re-interpolated along the spine into 8 markers, the time evolution of which is provided in `models/a2sw5_cycle_smoothed.csv`.

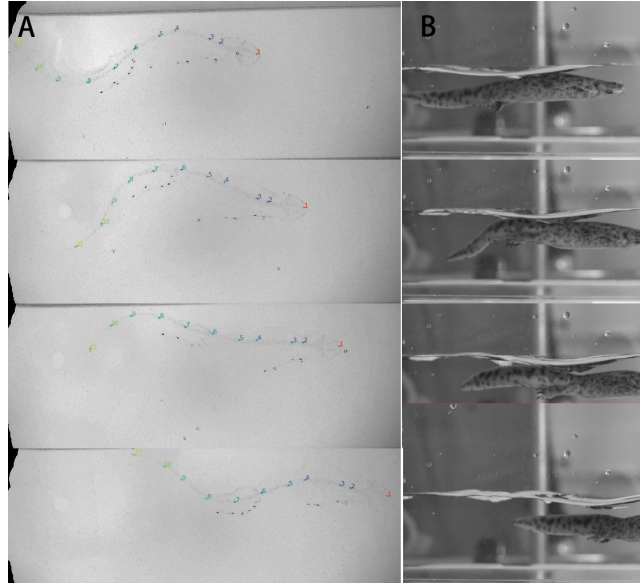


Figure 4: A. Recorded swimming sequences of an adult female Salamander *Pleurodeles Watl* under X-ray B. Synchronized optical recording of swimming experiment.

Note that the robot is scaled up by a factor of 6.5 compared to the animal subject. To match their physical interactions with the environments, we need to apply a dynamical scaling [4]. For example, an equivalent robot frequency can be scaled as:

$$f_{robot} = \sqrt{\frac{l_{salamander}}{l_{robot}}} \cdot f_{salamander} \quad (11)$$

Question 2.3 Analyze the provided animal data and compare the animal locomotion performance with your implemented controller in `cmc_controllers/exercise2.3.py`. Back up your analysis with metrics.

BONUS: Tune the controller to imitate the animal swimming behavior. Plot the animal kinematics and robot kinematics with quantified errors. (The bonus question would add to the project point if) properly solved. But the point cannot exceed the maximum possible point for Project 1)

2.3 Sensory feedback

In this part, you will add stretch feedback to the open-loop CPG implemented in the previous section.

Stretch feedback is a form of local proprioceptive feedback from stretch-sensitive edge cells distributed along the spinal cord of the animal. Such sensory cells have been identified and described in the locomotor circuits of lamprey [5] and zebrafish [6]. In the case of the robot (Polymander) used in this project, stretch feedback is obtained from encoders mounted on each actuated servo motor. We simplify the model by considering only ipsilateral excitatory projections (i.e., stretch feedback activates neurons on the same side of the spinal cord).

We consider only stretch and ignore compression in this project.

$$\vartheta_i(\text{left}) = \begin{cases} \theta_i, & \text{if } \theta_i > 0 \text{ (stretch)} \\ 0, & \text{otherwise} \end{cases} \quad (12)$$

$$\vartheta_i(\text{right}) = \begin{cases} -\theta_i, & \text{if } -\theta_i > 0 \text{ (stretch)} \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

which can be simplified as:

$$\vartheta_i(\text{left}) = \max(0, \theta_i) \quad (14)$$

$$\vartheta_i(\text{right}) = \max(0, -\theta_i) \quad (15)$$

The stretch feedback s_i for oscillator i is computed as the product of the stretch value ϑ_i and the feedback strength w^{ipsi} . To reduce the number of parameters, we use the same feedback strength for all segments:

$$s_i = w^{\text{ipsi}} \vartheta_i \quad (16)$$

After including stretch feedback, the oscillator equations are updated as:

$$\dot{r}_i = a(R_i - r_i) + s_i \cos(\theta_i) \quad (17)$$

where r_i is the oscillator amplitude, R_i the nominal amplitude, and θ_i the oscillator phase.

$$\dot{\theta}_i = 2\pi f_i + \sum_j r_j w_{ij} \sin(\theta_j - \theta_i - \phi_{ij}) - \frac{s_i}{r_i} \sin(\theta_i) \quad (18)$$

where f_i is the oscillator frequency, w_{ij} the coupling weights, and ϕ_{ij} the phase bias.

Question 3.1 Update `cmc_controllers/CPG_controller.py` to include stretch feedback. Run one simulation with feedback $w^{\text{ipsi}} = 3.0$ and another with $w^{\text{ipsi}} = 0$ using `scripts/exercise_3.1.py`, and explain why it is interesting to vary these values.

Plot the time evolution of oscillator states (θ and r), the sum and difference of muscle outputs ($M_{L_i} + M_{R_i}$ and $M_{L_i} - M_{R_i}$), body joint angles, and the robot CoM trajectory in 2D. Compute the neural frequencies, neural amplitudes, forward speed, and CoT. Analyze how adding stretch feedback changes the simulation performance compared to the open-loop CPGs in [Question 2.1](#).

Important: Implement stretch feedback as an optional CPG parameter. Ensure that the implementation of the CPG from previous exercises (without stretch feedback) remains functional.

Hint: To reduce errors in neural metric computation, skip the transient phase (i.e., initial signals before reaching steady state). Increase the number of simulation steps and set the buffer size to 20001 for better FFT resolution.

Question 3.2 Test different values of the ipsilateral feedback weight w^{ipsi} in the range $[-3.0, 17.0]$ using `scripts/exercise_3.2.py`. How does stretch feedback affect CoT, forward speed, neural frequency, and neural amplitude?

To model imperfections and noise in the neural network, we introduce neural disruptions to the robot controller as described in [\[7\]](#) (Fig. 3). Disruptions can be applied as **muted sensors**, **removed couplings**, or a combination of both.

The disruption is applied in the following steps:

- Implement a disruption mask: create an array with the same shape as the disrupted objects, with values sampled uniformly between 0 and 1.
- Create a boolean mask to select disrupted elements where the mask value is smaller than the disruption probability $p_{\text{dist_muted_sensor}}$ or $p_{\text{dist_removed_coupling}}$. For simplicity, keep the disruption targets fixed throughout the simulation.

- For muted sensors, set the sensor values to zero.
- For removed couplings, set the caudal and rostral coupling strengths to zero (keep the contralateral coupling).

To study the robustness and redundancy of ipsilateral coupling in the CPG network, we carry out an ablation study with two controller configurations (Tab. 3) and five disruption probabilities equally distributed between 0% and 15% (Tab. 2).

Name	<i>disruption_p_sensors</i>	<i>disruption_p_couplings</i>
Muted Sensors	[0, 15%]	0
Removed Couplings	0	[0, 15%]
Mixed	[0, 15%]	

Table 2: Different disruption configurations

Name	Sensory Feedback	Ipsilateral Coupling
Combined	+	+
Decoupled	+	-

Table 3: Different setups for the CPG controllers

Question 3.3 Implement the neural disruptions in `scripts/exercise_3.3.py`. Set the amplitude offset $c_{R_i,0}$ to 0 and the feedback strength w_{ipsi} to 10.0. Run a longer simulation with 10,001 steps (40s). Explore the effect of disruption by simulating mixed disruption configurations where `disruption_p_sensors` and `disruption_p_couplings` are both set to 20%. What do you observe in the simulation? How do the robot joint angles and trajectory compare to the case without disruption?

For both the combined and decoupled network setups in Tab. 3, run simulations with five disruption probabilities as defined in Tab. 2. Plot the robot speed and CoT evolution.

Hint: For reproducible experiments, pass `random_seed` to your controller initialization. Repeat the simulations with different seeds to obtain more general conclusions.

References

- [1] Q. Fu, Lee, *et al.*, “Polymander ii: an amphibious salamander-inspired robot with contact and flow sensors,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026. Accepted for publication.
- [2] A. J. Ijspeert, A. Crespi, D. Ryczko, and J.-M. Cabelguen, “From swimming to walking with a salamander robot driven by a spinal cord model,” *Science*, vol. 315, no. 5817, pp. 1416–1420, 2007.
- [3] A. L. Camp, T. J. Roberts, and E. L. Brainerd, “Swimming muscles power suction feeding in largemouth bass,” *Proceedings of the National Academy of Sciences*, vol. 112, no. 28, pp. 8690–8695, 2015.
- [4] K. Karakasiliotis, R. Thandiackal, K. Melo, T. Horvat, N. K. Mahabadi, S. Tsitkov, J.-M. Cabelguen, and A. J. Ijspeert, “From cineradiography to biorobots: an approach for designing robots to emulate and study animal locomotion,” *Journal of The Royal Society Interface*, vol. 13, no. 119, 2016.
- [5] S. Grillner, T. Williams, and P.-Å. Lagerbäck, “The edge cell, a possible intraspinal mechanoreceptor,” *Science*, vol. 223, no. 4635, pp. 500–503, 1984.

- [6] L. D. Picton, M. Bertuzzi, I. Pallucchi, P. Fontanel, E. Dahlberg, E. R. Björnfors, F. Iacoviello, P. R. Shearing, and A. El Manira, “A spinal organ of proprioception for integrated motor action feedback,” *Neuron*, vol. 109, pp. 1188–1201.e7, Apr. 2021.
- [7] R. Thandiackal, K. Melo, L. Paez, J. Herault, T. Kano, K. Akiyama, F. Boyer, D. Ryczko, A. Ishiguro, and A. J. Ijspeert, “Emergence of robust self-organized undulatory swimming based on local hydrodynamic force sensing,” *Science robotics*, vol. 6, no. 57, p. eabf6354, 2021.